

COMPE 510 – PA 3 Report

Rhilo Sotto
9/28/2025

Results

Part 1: Data Preprocessing

Fprintf result of part 1

```
===== Part 1: Data Preprocessing =====  
Loading data ...  
First 10 examples from the training dataset:  
x = [7 168 88 42 321 38 1 40], y = 1  
x = [1 103 30 38 83 43 0 33], y = 0  
x = [3 148 66 25 0 32 0 22], y = 0  
x = [4 114 65 0 0 22 0 37], y = 0  
x = [0 126 84 29 215 31 1 24], y = 0  
x = [3 102 74 0 0 30 0 32], y = 0  
x = [4 95 70 32 0 32 1 24], y = 0  
x = [2 74 0 0 0 0 0 22], y = 0  
x = [4 132 0 0 0 33 0 23], y = 1  
x = [6 129 90 7 326 20 1 60], y = 0  
  
Normalizing Features ...  
First 10 examples from the training dataset after normalization:  
x = [1 1 1 1 2 1 1 1], y = 1  
x = [-1 -1 -2 1 0 1 -1 -0], y = 0  
x = [-0 1 -0 0 -1 0 -1 -1], y = 0  
x = [0 -0 -0 -1 -1 -1 -0 0], y = 0  
x = [-1 0 1 1 1 -0 0 -1], y = 0  
x = [-0 -1 0 -1 -1 -0 -1 -0], y = 0  
x = [0 -1 0 1 -1 0 0 -1], y = 0  
x = [-1 -2 -4 -1 -1 -4 -1 -1], y = 0  
x = [0 0 -4 -1 -1 0 -1 -1], y = 1  
x = [1 0 1 -1 2 -2 0 2], y = 0  
  
Program paused. Press enter to continue.
```

Part 2: Maximum Likelihood & Gradient Ascent

Fprintf result of part 2

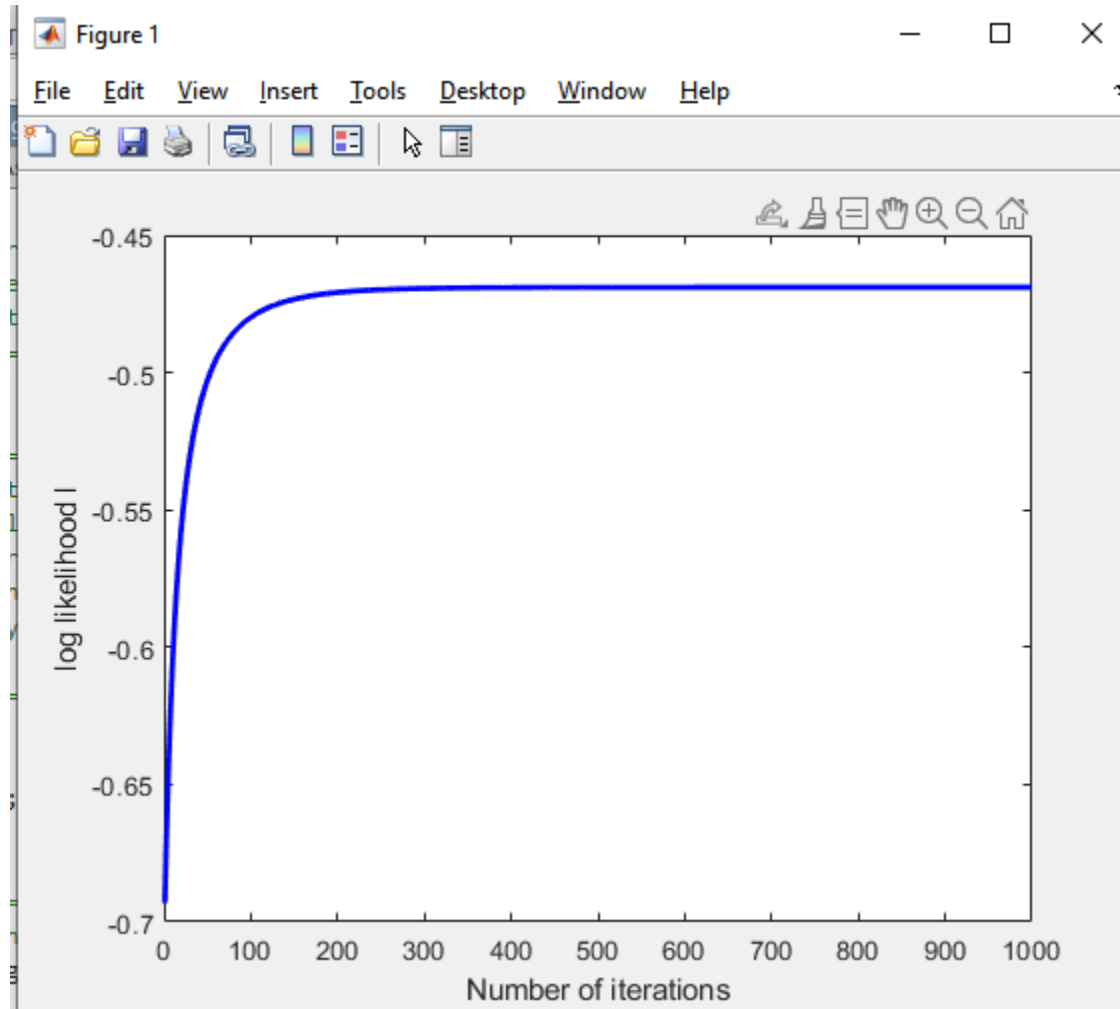
```
==== Part 2: Maximum Likelihood & Gradient Ascent ====  
beta computed from gradient descent:  
-0.594944  
0.213984  
0.821650  
-0.096774  
-0.005749  
0.041479  
0.482996  
0.198568  
0.250583  
  
Program paused. Press enter to continue.
```

Part 3: Evaluate Performance

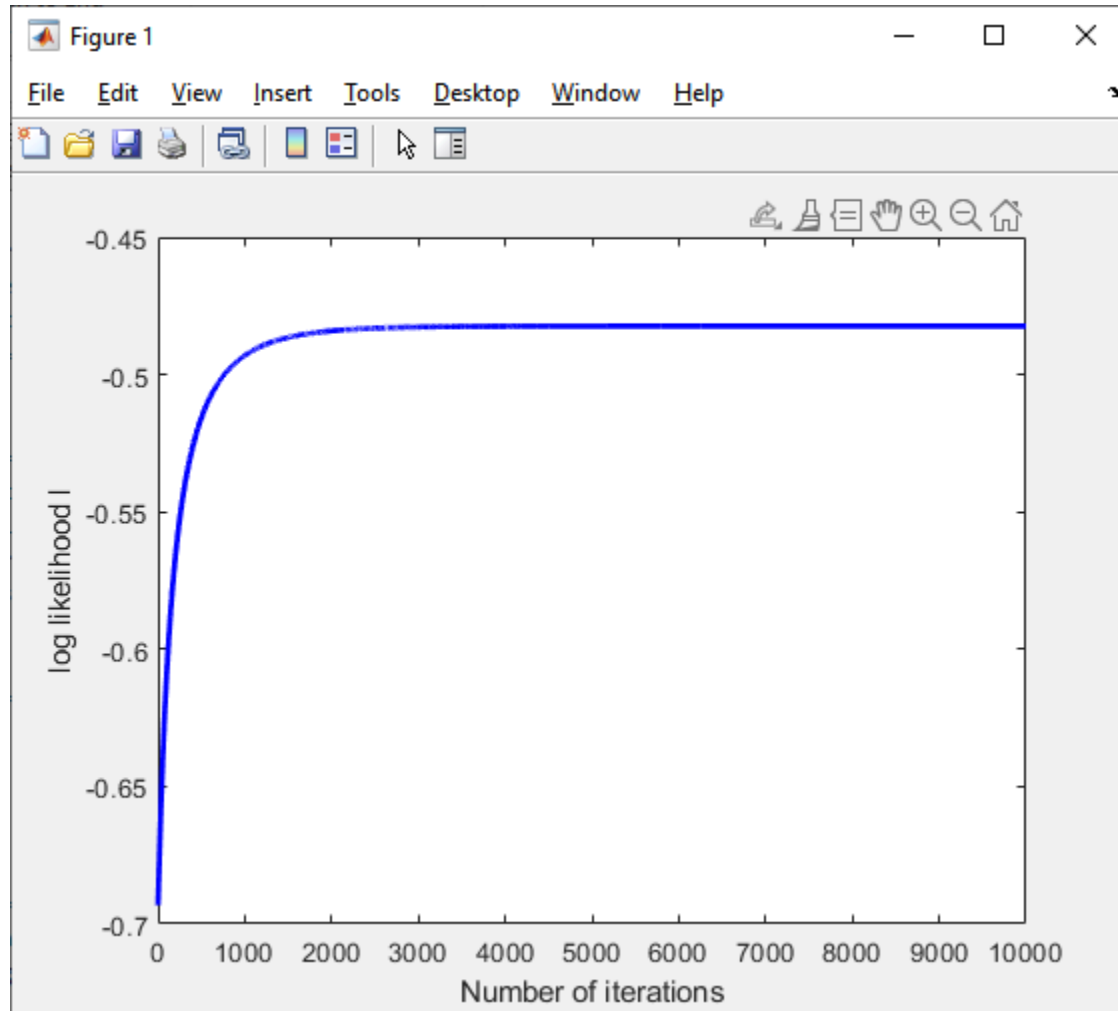
Fprintf result of part 3

```
==== Part 3: Evaluate Performance ====  
Accuracy:  
78.260870  
  
Predicted output:  
0  
>>
```

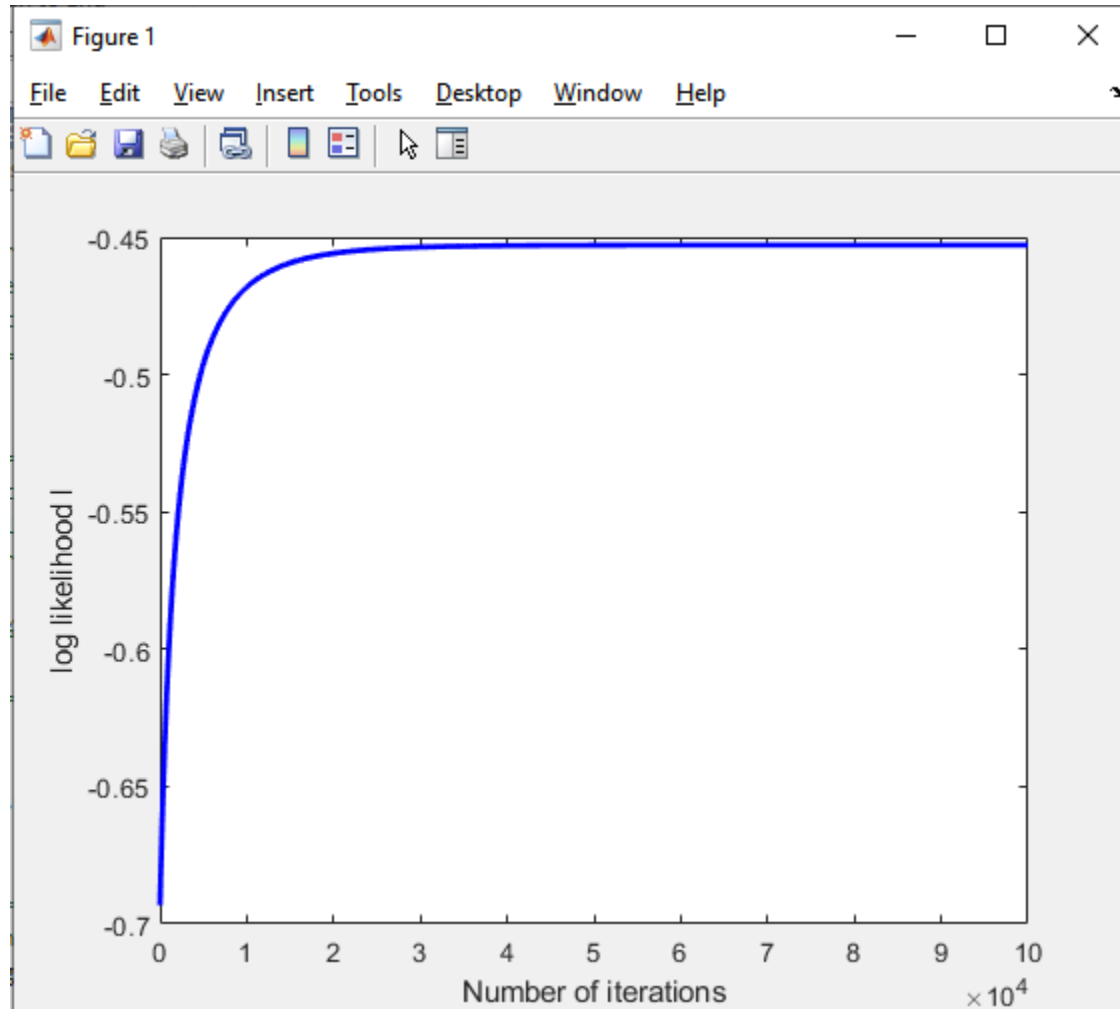
Graph with $\alpha = 0.1$, $\text{num_iters} = 1000$



Graph with $\alpha = 0.01$, num_iters = 10000



Graph with $\alpha = 0.001$, $\text{num_iters} = 100000$



Questions:

1. The learning rate determines whether the model will converge or not, as well as the amount of iterations it may take. With a learning rate that is too high, there is a chance the model won't converge, and with a learning rate that is too low, it will take a significant amount of iterations. It appears that a learning rate of 0.01 worked best for the data, because it managed to approach convergence in significantly fewer iterations, around 200, compared to 0.01 and 0.001 which took around 2000 and 20000 respectively.
2. The log-likelihood function in logistic regression is a measurement of how well the model's predicted probabilities match the binary outcomes, either a 0 or 1. Log-likelihood is more appropriate here because the output is a binary outcome, either having diabetes or not, if the output matches the outcome then the model is doing a good job.
3. If the model had high confidence such as 90% towards having diabetes but not actually having it, then it may be due to one feature in the data being too dominant and the model overfitting to it. This would lead to overconfidence, it could also be caused by fitting to noise or outliers in the data. An improvement would be flagging outliers to prevent overfitting.
If the model was uncertain but correct, having a low confidence such as 55% but correctly predicting diabetes, there may not be enough data or logistic regression is too simple to capture the true label. An improvement would be collecting more data.

Implementation

Implementing the sigmoid function went well, getting more used to the Matlab syntax. Using a reasonable amount of iterations didn't go well, I had too many iterations initially for each test case which resulted in each debug session taking a long time to calculate and run.